

Formal Methods and Specification

Assignment 4

Exercise 1

Consider the following input/output specification and program (all variables range over the natural numbers including zero):

- Input: x, y , s.t. $y \geq 1$
- Output: xy

```
assume ...
 $r \leftarrow x$ 
 $i \leftarrow 1$ 
while  $i < y$  do
  @
   $r \leftarrow r + x$ 
   $i \leftarrow i + 1$ 
@ ...
return  $r$ 
```

- Replace the dots in the program by formulas in such a way that the assertion on these lines check the requirement on the both the input and the output from the input/output-specification above. Note that the requirement on the output only contains a *term*, and not a *formula*¹. So you will have to extend the term to a formula that appropriately constrains the return value r of the program.
- Write down a table showing the values of the variables r and i at the position of the missing loop invariant at all loop iterations for some non-trivial input x and y .
- Come up with a guess for the missing loop invariant, write down the corresponding basic paths, and check whether the basic paths are correct. An informal check suffices, you do not have to prove anything.

If all basic paths are correct, the algorithm is correct. If not, you can (if you want) correct the invariant, and continue with item (c) of this exercise until all basic paths are correct.

(0.5 + 0.5 + 1.5 points)

¹If you do not remember how precisely a “term” is defined in logic, look this up!

Exercise 2

```

 $a_{in} \leftarrow a;$ 
 $l \leftarrow 0; u \leftarrow n - 1$ 
while  $l \leq u$  do
  @  $perm(a_{in}, a, n)$ 
  if  $a[l] \leq 0$  then  $l \leftarrow l + 1$ 
  if  $a[u] > 0$  then  $u \leftarrow u - 1$ 
  if  $l \leq u \wedge a[l] > 0 \wedge a[u] \leq 0$  then
     $al \leftarrow a[l]; au \leftarrow a[u]$ 
     $a[l] \leftarrow au; a[u] \leftarrow al$ 
  @  $perm(a_{in}, a, n)$ 
@  $perm(a_{in}, a, n) \wedge \exists k . part(a, n, k)$ 

```

Here, the predicate $part(a, n, k)$ holds for k that partition the array a into numbers less or equal zero and number greater zero:

$$\forall a \in \mathcal{A} \forall k . part(a, n, k) :\Leftrightarrow [\forall i \in \{0, \dots, k-1\} . a[i] \leq 0] \wedge [\forall i \in \{k, \dots, n-1\} . a[i] > 0]$$

And the predicate $perm(a, b, n)$ expresses that the elements $0, \dots, n-1$ of the arrays a and b contain the same elements, potentially in different order:

$$perm(a, b, n) :\Leftrightarrow \begin{array}{l} \exists c . permnum(c, n) \wedge \\ \forall i \in \{0, \dots, n-1\} . b[c[i]] = a[i] \end{array}$$

$$permnum(a, n) :\Leftrightarrow \begin{array}{l} \forall i \in \{0, \dots, n-1\} \exists j \in \{0, \dots, n-1\} . a[j] = i \wedge \\ \forall i, j \in \{0, \dots, n-1\} . i \neq j \Rightarrow a[i] \neq a[j] \end{array}$$

- (a) Try to find a program input, for which the last assertion of the program results in an assertion failure. If you cannot find such an input, think of an informal, intuitive argument for why the program is correct. If you found such an input, correct the problem using a change that is as small as possible, and again think of an informal, intuitive argument for why the program is correct now.
- (b) On the next page, you will find all 12 basic paths of this program. For each basic path, check whether it is correct. Just answer either “yes” or “no” and informally explain why. So no proofs are necessary.
- (c) Replace the last assertion of the program by an assertion that does not only claim existence of a k , but explicitly provides its value. For this, find a term t such that after replacing the asserted formula by $perm(a_{in}, a, n) \wedge part(a, n, t)$, there is no program execution for which this assertion fails.
- (d) Extend the assertions within the loop in such a way that all basic paths are correct. Again, no proofs are necessary.

(1 + 1 + 1 + 1.5 points)

$a_{in} \leftarrow a$
 $l \leftarrow 0; u \leftarrow n - 1$
assume $l \leq u$
 $@ \text{perm}(a_{in}, a, n)$

assume $\text{perm}(a_{in}, a, n)$
assume $l \leq u$
 $@ \text{perm}(a_{in}, a, n)$

assume $\text{perm}(a_{in}, a, n)$
assume $a[l] \leq 0$
 $l \leftarrow l + 1$
assume $a[u] > 0$
 $u \leftarrow u - 1$
assume $l \leq u \wedge a[l] > 0 \wedge a[u] \leq 0$
 $al \leftarrow a[l]; au \leftarrow a[u];$
 $a[l] \leftarrow au; a[u] \leftarrow al$
 $@ \text{perm}(a_{in}, a, n)$

assume $\text{perm}(a_{in}, a, n)$
assume $\neg a[l] \leq 0$
assume $a[u] > 0$
 $u \leftarrow u - 1$
assume $l \leq u \wedge a[l] > 0 \wedge a[u] \leq 0$
 $al \leftarrow a[l]; au \leftarrow a[u];$
 $a[l] \leftarrow au; a[u] \leftarrow al$
 $@ \text{perm}(a_{in}, a, n)$

assume $\text{perm}(a_{in}, a, n)$
assume $a[l] \leq 0$
 $l \leftarrow l + 1$
assume $\neg a[u] > 0$
assume $l \leq u \wedge a[l] > 0 \wedge a[u] \leq 0$
 $al \leftarrow a[l]; au \leftarrow a[u];$
 $a[l] \leftarrow au; a[u] \leftarrow al$
 $@ \text{perm}(a_{in}, a, n)$

assume $\text{perm}(a_{in}, a, n)$
assume $\neg a[l] \leq 0$
assume $\neg a[u] > 0$
assume $l \leq u \wedge a[l] > 0 \wedge a[u] \leq 0$
 $al \leftarrow a[l]; au \leftarrow a[u];$
 $a[l] \leftarrow au; a[u] \leftarrow al$
 $@ \text{perm}(a_{in}, a, n)$

$a_{in} \leftarrow a$
 $l \leftarrow 0; u \leftarrow n - 1$
assume $\neg l \leq u$
 $@ \text{perm}(a_{in}, a, n) \wedge \exists k . \text{part}(a, n, k)$

assume $\text{perm}(a_{in}, a, n)$
assume $\neg l \leq u$
 $@ \text{perm}(a_{in}, a, n) \wedge \exists k . \text{part}(a, n, k)$

assume $\text{perm}(a_{in}, a, n)$
assume $a[l] \leq 0$
 $l \leftarrow l + 1$
assume $a[u] > 0$
 $u \leftarrow u - 1$
assume $\neg[l \leq u \wedge a[l] > 0 \wedge a[u] \leq 0]$
 $@ \text{perm}(a_{in}, a, n)$

assume $\text{perm}(a_{in}, a, n)$
assume $\neg a[l] \leq 0$
assume $a[u] > 0$
 $u \leftarrow u - 1$
assume $\neg[l \leq u \wedge a[l] > 0 \wedge a[u] \leq 0]$
 $@ \text{perm}(a_{in}, a, n)$

assume $\text{perm}(a_{in}, a, n)$
assume $a[l] \leq 0$
 $l \leftarrow l + 1$
assume $\neg a[u] > 0$
assume $\neg[l \leq u \wedge a[l] > 0 \wedge a[u] \leq 0]$
 $@ \text{perm}(a_{in}, a, n)$

assume $\text{perm}(a_{in}, a, n)$
assume $\neg a[l] \leq 0$
assume $\neg a[u] > 0$
assume $\neg[l \leq u \wedge a[l] > 0 \wedge a[u] \leq 0]$
 $@ \text{perm}(a_{in}, a, n)$

Exercise 3

- (a) Write down a first-order predicate logical formula that represents the following I/O-specification in the way discussed in the lecture on functions and procedures:

function $s(a, k)$
Input: $a \in \mathcal{A}, k \in \mathcal{N}$ s.t. $k \geq 1$
Output: $1 + \sum_{i \in \{0, \dots, k-1\}} a[i]$

- (b) Add the formula from the previous item as an assumption to the following program, and write down the resulting verification condition.

assume $\forall i . p[i] \geq 4$
 $r \leftarrow s(p, 3)$
@ $r \geq 9$

Here, p is an array of integers, and all other variables are integers.

- (c) Check, whether the verification condition from the previous item holds. You do not have to prove anything, you just should be able to explain why it holds/does not hold.
- (d) Prove the verification condition, following the lecture on handling function calls. Here, you may freely use whatever knowledge you have about arrays and integers (i.e., not only the axioms of the corresponding logical theories).

(1+0.5+0.5+1 points)

Exercise 4

Consider the class

```
class P
constructor init( $x \in \mathcal{N}, y \in \mathcal{N}$ )
method add( $x \in \mathcal{N}$ )
method fst():  $\mathcal{N}$ 
method snd():  $\mathcal{N}$ 
```

where the notation used is the object-oriented notation used in the lecture on abstract data types. Now translate the following basic path from object-oriented notation to procedural notation (i.e., notation without dots).

```
 $x \leftarrow \text{init}(0, 1)$ 
 $y \leftarrow x$ 
 $x.\text{add}(7)$ 
 $y.\text{add}(1)$ 
assume  $x.\text{fst}() = y.\text{snd}()$ 
@  $x.\text{snd}() < 10$ 
```

// after this assignment, y will be a *copy* of x ,
// so this modification will *not* change y ,
// and this modification will *not* change x .

(1 points)

Exercise 5

```
assume  $r = 0 \wedge s = 0$   
for  $i \leftarrow 1$  to  $n$  do  
  @  $\top$   
  if  $a[i] < 0$  then  $r \leftarrow r + 1$   
  @  $r = |\{k \in \{1, \dots, i\} \mid a[k] \neq 0\}|$   
  if  $a[i] > 0$  then  $s \leftarrow s + 1$   
  @  $s = |\{k \in \{1, \dots, i\} \mid a[k] \neq 0\}|$   
@  $r + s = |\{k \in \{1, \dots, n\} \mid a[k] \neq 0\}|$ 
```

- (a) For each verification condition of the program, write down whether it holds or not, and if not, also note the reason why. Apart from that, you do not have to write down anything, neither verification conditions themselves nor proofs.
- (b) Change the assertions in the program in such a way that all verification conditions hold. Again, you do not have to prove anything. Still check *each* verification condition in an informal way.

Do *not* change the program itself.

(1.5+2 points)